

Bridging the Gap: Extended Game Development Projects for Student Employability and Experiential Learning

Drew A. Clinkenbeard
dclinkenbeard@csumb.edu

California State University, Monterey Bay
Seaside, California, USA

Bryan Berry
bberry@csumb.edu

California State University, Monterey Bay
Seaside, California, USA

Abstract

This work-in-progress paper examines a 14-week intensive summer research project designed to bridge the gap between academic preparation and industry requirements in game development. The project engaged four undergraduate computer science students at California State University, Monterey Bay, in authentic roguelite game development, combining experiential learning theory with professional software development practices. Students developed a Unity-based game featuring core combat mechanics, procedurally generated content, and custom art assets while learning collaborative workflows through Git/GitHub, code review practices, and agile methodologies. Preliminary results demonstrate substantial student learning across technical, professional, and metacognitive domains. Analysis reveals four distinct developmental phases enabled by the extended timeline: foundation building, core feature development, iteration and refinement, and architectural maturity. Student self-assessments document significant confidence growth, technical skill acquisition, and transformation from task completion to creative ownership. The project validates Kolb's Experiential Learning Theory in action and provides empirical evidence that extended timelines produce qualitatively different outcomes than typical semester projects. Findings suggest implications for curriculum design, summer research program models, and employability interventions at teaching-focused institutions serving workforce-bound students.

CCS Concepts

• **Social and professional topics** → **Computing education**; • **Computing methodologies** → *Procedural animation*.

Keywords

experiential learning, game-based learning, procedural content generation, project-based learning, computer science education, student employability, wave function collapse

ACM Reference Format:

Drew A. Clinkenbeard and Bryan Berry. 2026. Bridging the Gap: Extended Game Development Projects for Student Employability and Experiential

Learning. In *Proceedings of Foundations of Digital Games 2026 (FDG '26)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

1.1 Problem Statement and Motivation

The video game industry represents one of the fastest-growing sectors in the technology economy, with the US gaming market projected to expand from \$63.36 billion in 2025 to \$99.33 billion by 2030 [7]. This growth has created substantial demand for skilled game developers, yet a significant gap exists between academic preparation and industry requirements. Entry-level positions require both strong portfolios and prior development experience, creating a paradoxical barrier: students need experience to get hired but need to be hired to gain experience. Traditional computer science curricula, while providing solid theoretical foundations, often fail to address these practical employability requirements.

1.2 Current State of CS Education

Most undergraduate computer science programs incorporate group projects for practical development experience, but semester-based calendars impose significant time constraints. Typical projects span five weeks or less, severely limiting depth and complexity. This compressed timeline forces students to focus on rapid implementation rather than thoughtful design, testing, or iterative refinement.

1.3 The Extended Project Solution

This work-in-progress paper examines an alternative approach: a 14-week intensive research project providing authentic game development experience, allowing students to engage with the full software development life cycle.

1.4 Research Questions

This work-in-progress paper explores the following questions:

- (1) How does an extended timeline (14 weeks) affect student learning outcomes compared to projects lasting 5 weeks or less?
- (2) What technical and professional skills do students develop through authentic game development?
- (3) What challenges arise in mentoring students through complex algorithm implementation?
- (4) How can this model be replicated at similar institutions?

1.5 Paper Organization

Section 2 reviews relevant literature on experiential learning theory, game-based learning in computer science, project-based learning

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FDG '26, Location TBD

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/05
<https://doi.org/XXXXXXX.XXXXXXX>

and employability, procedural content generation, and summer research programs. Section 3 describes the project design and methodology. Section 4 presents preliminary results. Section 5 discusses the interpretation of these findings. Section 6 outlines next steps and future research directions. Section 7 concludes with broader implications for computer science education.

2 Literature Review

This review examines the theoretical foundations and empirical evidence supporting extended game development projects as vehicles for experiential learning in computer science education. We organize our discussion around six key areas: experiential learning theory, game-based learning effectiveness in CS, project-based learning and employability outcomes, procedural content generation techniques, the Wave Function Collapse algorithm, and summer research programs. We conclude by identifying critical gaps this work addresses.

2.1 Experiential Learning Theory

Kolb's Experiential Learning Theory (1984) provides the foundational framework for understanding how students learn through direct experience [5]. The theory posits a four-stage cycle: concrete experience, reflective observation, abstract conceptualization, and active experimentation. Central to Kolb's model is the premise that "knowledge is created through the transformation of experience," emphasizing that effective learning requires learners to progress through all four stages in a continuous spiral [5]. Recent applications demonstrate continued relevance to computer science education. Konak et al. (2014) showed that activities designed around all four stages increased student interest and competency compared to traditional tutorials [6]. Morris (2020) identified that authentic learning requires active participation, situated knowledge, novel experiences involving risk, and inquiry into real-world problems [10], characteristics aligning with extended game development projects.

2.2 Game-Based Learning in Computer Science

Empirical evidence for game-based learning (GBL) effectiveness in computer science education has grown substantially, with multiple meta-analyses providing quantitative support. Zeng et al. (2024) conducted a comprehensive meta-analysis of 41 studies with 49 independent samples involving over 5,071 participants spanning 2008 to 2023, demonstrating a significant large effect size ($g = 0.822$) for gamification's impact on academic performance [13]. De Almeida Rodrigues et al. (2022) analyzed 21 empirical studies and found that gamification had the largest effect on student motivation, followed by academic achievement, while having the least effect on cognitive load [1]. Notably, their analysis revealed that effects were more pronounced in text-based programming environments compared to graphical programming contexts, suggesting that GBL may be particularly valuable when students face steeper learning curves.

Videnovik et al. (2023) examined 113 articles published between 2017 and 2021, revealing that programming remains the dominant CS topic addressed through GBL [12]. Critically, most research focuses on learning by playing existing games rather than on students designing games themselves—a gap this work addresses.

However, critical perspectives temper these findings. Dichev and Dicheva (2017) cautioned that short-term engagement gains may not translate to sustained outcomes [2], while Mekler et al. (2017) showed gamification improves performance but not intrinsic motivation [9]. This distinction becomes particularly important when designing extended projects requiring sustained motivation over weeks or months. Foundational theoretical work by Papert (1980) and Gee (2003) suggests that games are not merely motivational tools but learning environments that naturally embody effective pedagogical principles including active learning, immediate feedback, and construction of personally meaningful artifacts [3, 11].

2.3 Project-Based Learning and Student Employability

Project-based learning (PBL) has demonstrated significant impact on students' job readiness and career preparation, particularly when projects incorporate authentic industry practices. A 2024 study on industry-enriched PBL documented a 25% increase in job-related competencies when students engaged with authentic industry practices, alongside 30% increases in engagement and 35% increases in course satisfaction [?]. This research identified an optimal threshold of approximately 40% PBL content integrated with industry-standard tools and workflows.

In game development, entry-level positions typically require both a strong portfolio and prior development experience, creating a paradoxical barrier. Extended research projects that produce portfolio-quality artifacts can bridge this gap by providing authentic development experiences mirroring industry expectations.

Kissel and Stuetzle (2020) proposed an experiential learning framework specifically designed for smaller computer science programs, recognizing that not all institutions have the resources of major research universities [4]. Their framework emphasizes practical, implementable approaches to providing authentic experiences at institutions similar to regional comprehensive universities. This work is particularly relevant for understanding how summer research programs can serve as vehicles for experiential learning at teaching-focused institutions.

2.4 Procedural Content Generation in Games

Procedural content generation (PCG) has become a foundational technique in modern game development, enabling creation of varied game content through algorithmic means rather than manual authoring. Maleki and Zhao (2024) conducted a comprehensive survey of PCG techniques, positioning Wave Function Collapse within the broader landscape of constraint-based generation methods [8]. From a pedagogical perspective, PCG represents a rich domain for computer science learning, requiring students to integrate knowledge across data structures, constraint satisfaction algorithms, software architecture, and debugging strategies for stochastic systems.

Wave Function Collapse (WFC) offers particular advantages as a learning vehicle. The algorithm defines tiles with adjacency rules specifying permitted neighbors, then iteratively places tiles by selecting and placing one tile (often randomly), constraining neighboring positions based on adjacency rules, and repeating until the space is filled. When contradictions occur, the algorithm backtracks.

This approach suits roguelite level generation by producing locally coherent structures while maintaining global variety. WFC synthesizes multiple CS concepts including two-dimensional data structures, constraint propagation algorithms, backtracking and recursion, and probabilistic selection strategies. The algorithm allows deep customization through tile design and rule specification, requiring iterative refinement, making it pedagogically valuable as students progress from implementation to creative application.

2.5 Summer Research Programs in Computer Science

Summer undergraduate research programs have become a standard mechanism for providing students with intensive, mentored research experiences, though most programs focus on preparing students for graduate school rather than industry careers. Traditional summer research experiences emphasize scientific inquiry, publication, and academic skill development. However, the Council on Undergraduate Research recognizes that undergraduate research can serve multiple purposes, including career preparation for students entering the workforce directly after graduation.

Extended summer projects offer several advantages over traditional semester-based courses. The intensive format provides sustained engagement with a single complex problem, enabling students to work through multiple iterations of design, implementation, testing, and refinement. This timeline more closely approximates authentic software development cycles than typical 5-week course projects, which often compress development into a sprint prioritizing rapid implementation over thoughtful design.

Summer research programs provide dedicated mentorship and resources that may not be feasible during the academic year. Faculty mentors can work closely with small teams, providing scaffolded guidance that gradually transfers responsibility as competence develops. This apprenticeship model aligns with experiential learning theory and authentic practice in software development communities.

2.6 Gaps in Literature and Our Contribution

This work addresses several significant gaps in CS education research. While substantial evidence supports game-based learning through playing games, few studies examine learning through game development itself [12]. Most project-based learning research examines compressed 5-week semester projects, limiting insights into how extended timelines affect learning depth and quality. Our 14-week intensive summer format provides empirical evidence about student learning outcomes when game development serves as the primary learning activity over an extended period while examining PCG's pedagogical potential. We explicitly address employability gaps by producing portfolio-quality artifacts and adapting summer research models to serve workforce-bound students at teaching-focused institutions.

3 Project Design and Methodology

3.1 Project Context

California State University, Monterey Bay (CSUMB) is a medium-sized Hispanic-Serving Institution within the California State University system, serving approximately 7,500 students on the central California coast. Founded in 1994, CSUMB emphasizes experiential learning and workforce preparation for a predominantly first-generation, socioeconomically diverse student population. The Computer Science and Information Technology department enrolls approximately 400 students, with many pursuing direct entry into the technology workforce.

This project is funded through an internal grant from CSUMB—the Research Scholarship and Creative Activity (RSCA) grant—intended to support student and faculty work on creative activities.

3.2 Theoretical Framework: Kolb's Experiential Learning Cycle

Kolb's Experiential Learning Cycle describes learning as transformation of experience through four iterative stages: concrete experience, reflective observation, abstract conceptualization, and active experimentation [5]. Agile software development practices embody this cycle structurally: sprints provide concrete experience with implementation, retrospectives enable reflective observation, planning sessions support abstract conceptualization of solutions, and subsequent iterations represent active experimentation with refined approaches.

3.3 Student Selection and Team Formation

The initial plan was to recruit a small team of two programmers and two artists. Students were selected based on their interest in game development and activity in the game development club on the CSUMB campus. Each student was evaluated based on previous work and recommendations from peers. Students participated in brief interviews to assess their willingness to work on the project and to establish time commitments.

3.4 Project Scope: Roguelite Game Development

This section discusses the rationale for the game genre, technology choices, key deliverables, and skills students will develop.

3.4.1 Genre Rationale. The roguelite genre was selected based on pedagogical and practical considerations. Roguelites are characterized by procedurally generated content, permanent death with meta-progression, and high replayability. The genre's reliance on PCG provides authentic context for implementing and testing algorithms, as procedural level generation is a core gameplay mechanic directly impacting player experience. Roguelites encourage modular design through component-based systems for items, enemies, and abilities, providing experience structuring complex interdependent systems. The iterative nature facilitates continuous playtesting, enabling feedback on design decisions and algorithm performance. Practically, the structure allows incremental development: core mechanics first, then procedural generation, meta-progression, and content variety in subsequent iterations. Finally, roguelites represent a commercially viable indie genre, making student work

relevant to portfolio development and employment while maintaining challenges appropriate for undergraduate research.

3.4.2 Technical Stack. The technical stack—Unity game engine, C# programming, and Git/GitHub for version control—balances pedagogical objectives with industry alignment. Unity is the most widely adopted game engine in indie and mobile development, making it directly relevant to career prospects while providing mature documentation and community support. Students had prior Unity exposure through coursework, allowing focus on advanced topics like procedural content generation. C# is a statically-typed, object-oriented language with modern features that reinforces core CS concepts while providing expressive power for complex game systems. Git and GitHub provide essential professional development tools: distributed version control workflows, collaborative code review through pull requests, and project management via Issues. The project's use of branching strategies, code review, and issue tracking mirrors authentic industry practices, cultivating professional software engineering habits.

3.4.3 Key Deliverables. The project aims to produce:

- (1) Vertical slice demo (December 2025) - core mechanics
- (2) Full demo with PCG (April 2026) - complete experience
- (3) Novel PCG algorithm using Wave Function Collapse
- (4) Lesson plans for teaching game design concepts in regular courses
- (5) Documentation for future students and replication

3.5 Learning Objectives

3.5.1 Technical Skills.

- C# programming proficiency
- Unity game engine mastery
- Algorithm design and implementation (PCG)
- Software architecture for games
- Version control (Git/GitHub)

3.5.2 Professional Skills.

- Agile development practices
- Team collaboration and communication
- Project scoping and time management
- Code review and quality standards
- Documentation and technical writing

3.5.3 Metacognitive Skills.

- Problem decomposition
- Debugging complex systems
- Learning from playtesting feedback
- Adapting to changing requirements

3.6 Development Process

3.6.1 Onboarding. An onboarding meeting introduced the project, tools, and initial timeline. Students were already familiar with Unity, so onboarding focused on the Git version control system and GitHub for organizing the project. Students were guided through creating GitHub issues, committing work, and submitting work for review. Once tools were established, the project goal was introduced along with expected game outcomes. The team then brainstormed ideas and established initial goals for the first week.

3.6.2 Weekly Cycle. After an initial in-person meeting, the team met weekly on Fridays using Discord. Weekly progress toward GitHub issues was discussed and updated as needed. Each week, students were encouraged to create pull requests for their code to be reviewed and merged into the main codebase. As the project progressed and new features were needed or bugs were discovered, refactoring occurred as students evaluated the code and suggested improvements. The game was regularly playtested internally to ensure it continued to function smoothly.

3.7 Procedural Content Generation Focus

A major goal of this work was developing a PCG algorithm following WFC to generate game content. This remains a work in progress as the initial vertical slice of the game features was required before it could be fully pursued.

3.7.1 Wave Function Collapse Implementation Plan. The Wave Function Collapse implementation is scheduled for the winter break between fall and spring semesters (January 2026), after the core game systems are stable. As described in the literature review, WFC's tile-based constraint satisfaction approach is well-suited to roguelite level generation, allowing students to apply foundational knowledge gained during the first project phase to a complex algorithmic challenge.

3.7.2 Learning Progression.

- (1) **Weeks 1-2:** Understand existing WFC implementations
- (2) **Weeks 3-5:** Adapt WFC to game's specific needs
- (3) **Weeks 6-8:** Optimize for performance, add constraints
- (4) **Weeks 9-10:** Polish, document, prepare demo

4 Preliminary Results

This section presents preliminary results collected through October 2025, documenting the first 14-week phase of this work-in-progress project. Data sources include GitHub repository metrics, student self-assessment reflections, faculty observations, and project artifacts.

4.1 Project Timeline and Milestones

The project ran from July 14 through October 17, 2025, spanning 14 weeks during the summer intensive period. The timeline required significant adjustments from original plans, most notably postponing Wave Function Collapse implementation from September to January 2026. This adjustment reflects authentic software development realities: foundational game systems required more development time than initially estimated, and students' learning curve with Unity and collaborative workflows necessitated extended onboarding. The Discussion section provides detailed analysis of how the extended timeline enabled distinct developmental phases critical to learning outcomes.

4.2 Technical Outcomes

By October 2025, the team achieved substantial technical progress across game development, collaborative workflows, and professional practices:

4.2.1 Game Development Progress.

- Core mechanics implemented: complete player movement, item spawn and pickup, weapon use
- Assets created: 50 custom art assets included in the project; both artists have created additional assets not yet included, totaling approximately 75 discrete assets
- Levels/areas completed: Two levels: the frame level where the player starts each run and one sample level
- Lines of code: 2,225 lines of C# code

4.2.2 PCG Algorithm Status. As of October 2025, the Wave Function Collapse implementation remains in the prototyping and research phase. The team prioritized developing a fully-featured playable demo with core mechanics, player movement, combat systems, and art asset integration before beginning PCG implementation. This represents a deliberate project management decision to establish a solid foundation for procedural generation rather than attempting to implement WFC algorithms without a stable game framework. The original timeline anticipated initial PCG implementation by September 2025, but this milestone was adjusted to accommodate the complexity of core game development and the team's learning curve. Students have begun researching WFC algorithms and studying existing implementations, with active development scheduled for winter break. This timeline adjustment reflects authentic software development realities where foundational systems must be stable before advanced features can be effectively implemented and tested.

4.2.3 GitHub Activity Metrics. The GitHub repository demonstrates active collaborative development. As of December 2025, the repository contains 81 commits by 4 team members, with 2,225 lines of C# code. The team utilized GitHub's collaborative features extensively: 15 pull requests (all merged) and 30 Issues (16 closed, 14 open). This reflects consistent engagement with professional practices including code review, task tracking, and distributed version control. The pull request to commit ratio (15:81) indicates most code changes underwent review before integration, demonstrating quality assurance practices. Issue tracking shows both feature planning and bug resolution workflows typical of authentic software projects.

4.3 Student Learning Outcomes

Student self-assessment documents collected in October 2025 reveal significant learning outcomes across technical, professional, and metacognitive domains. The four-member team consisted of two programmers and two artists, all of whom provided detailed reflections on their skill development, confidence growth, and collaborative experiences throughout the project.

4.3.1 Technical Skill Development. Student reflections demonstrate substantial technical skill acquisition. The lead programmer advanced from basic Unity familiarity to implementing complex systems, noting growth in C# proficiency and PCG concepts, specifically Wave Function Collapse. He transitioned from GUI-based to command-line Git, indicating deepening professional tool proficiency. Both artists reported significant learning curves in game-specific art production, with pixel art and animation representing entirely new domains. One described animation as "the biggest hurdle" but developed streamlined workflows after months of practice. Students emphasized cross-disciplinary communication skills,

particularly how to "communicate effectively with programmers" and "export art correctly" for Unity implementation. Learning extended beyond individual tools to complete production pipelines from asset creation through engine integration.

4.3.2 Professional Skill Development. All four students identified collaborative project management as a critical learning outcome. Students consistently highlighted GitHub issue tracking as essential for managing project complexity, with one noting that "being able to create issues and assign them to each other to keep track of every piece of development has made the challenge of developing a game a whole lot less stressful." Communication through Discord and in-person meetings emerged as foundational practices, with students reporting that regular team communication proved "very crucial and important" for coordinating distributed creative work. Students developed comfort with iterative design processes, with one artist noting a transformation from being "much more stubborn" about refusing to sketch or plan to recognizing that "the quality of the end product increases when you give yourself time and space to draft things." The ability to "start over when something isn't working" represented significant professional growth. Students also developed project scoping skills, learning to "manage the scope of a project and break it down into smaller, more digestible parts"; a metacognitive capability essential for managing complex software development.

4.3.3 Confidence Growth and Team Culture. Student reflections reveal substantial confidence development throughout the project timeline. The lead programmer candidly described entering the project without "the highest levels of confidence" but reported that confidence "has risen over the course of months" as he completed incremental project components and realized "I am capable of creating something that has been my lifelong dream, a video game." An artist reported that confidence levels "greatly improved," noting she was "no longer intimidated by asking questions." Students attributed confidence growth both to skill mastery and to supportive team culture. Multiple students emphasized the project's collaborative environment, describing it as "very supportive" and noting that "we are all still learning as we develop this game, so I never feel afraid to ask questions." One student characterized the team dynamic as working "with a group of highly passionate, motivated people," while another noted that "seeing everyone else's work pushes me to continue learning and growing." This peer learning environment appears to have reduced anxiety about skill gaps while fostering collective problem-solving approaches. Students reported developing comfort with asking for help, balancing individual visions with collaborative decision-making, and maintaining morale through shared accomplishment recognition.

4.3.4 Representative Student Reflections. Student reflections provide qualitative evidence of experiential learning outcomes. One programmer reflected: "Throughout the process of this project, up to the current point, I have seen how crucial it is to be able to work in a team to work towards one big goal. . . Each of us has been able to piggyback off of one another in order to learn how to work in a team for creating a game." An artist described the contrast with typical coursework: "This project has been a really great experience so far, and has shown me what a creative project feels like

when a group of people are passionate about and invested in the quality of the outcome.” Another artist emphasized collaborative skill development: “Throughout my time working on this project, I have made sure to communicate with my team members... sharing my progress and making sure to get feedback from everyone... It is important that everyone is aware of what progress is being made and that we all feel a sense of accomplishment, pride, and success throughout working on the project.” These reflections suggest that the extended project timeline and authentic collaborative context provided learning experiences qualitatively different from traditional course projects.

4.4 Challenges Encountered

As a work-in-progress project, documenting challenges honestly provides valuable insights for both understanding student learning processes and informing future implementations of similar extended research experiences. The challenges encountered span technical, project management, and learning domains, each revealing important considerations for designing authentic software development experiences in educational contexts.

4.4.1 Technical Challenges. The team encountered two primary technical challenges. Animation integration proved complex, requiring students to understand Unity’s animation system, sprite sheet organization, state machines, and integration between art assets and code-driven behaviors. Students addressed this through systematic research of documentation and tutorials, experimentation, and iterative refactoring. Code architecture presented ongoing challenges as the codebase grew. Students developed understanding of modular design, component-based architecture, and managing dependencies between systems.

4.4.2 Project Management Challenges. Timeline estimation emerged as a significant project management challenge, particularly regarding the procedural content generation implementation timeline. The team’s original plan anticipated PCG development beginning by September 2025, but the complexity of establishing core game mechanics, integrating art assets, and debugging fundamental systems required timeline adjustments. This challenge reflects authentic software development realities where initial estimates often underestimate the time required for foundational work, particularly when team members are simultaneously learning tools and practices. Task estimation difficulties were compounded by students’ limited prior experience with projects of this scope and duration, making it challenging to accurately predict how long implementation tasks would require. These challenges provided valuable learning experiences regarding project planning, scope management, and the importance of establishing stable foundations before implementing advanced features.

4.4.3 Learning Challenges. Students faced two primary learning challenges beyond technical skill acquisition. Confidence issues emerged as students grappled with the project’s scope and complexity, particularly when comparing their current capabilities to the ambitious end goals. The overwhelming nature of game development as a complex, multifaceted challenge occasionally caused students to question whether they could successfully complete the project. Additionally, the tension between learning new skills and

delivering functional components created pressure, as students needed to simultaneously acquire new technical knowledge while producing work that integrated with team members’ contributions. These challenges are inherent to authentic project-based learning but rarely surface in traditional coursework where exercises are carefully scoped to match existing student capabilities.

4.5 Solutions and Adaptations

The project employed multiple strategies to address challenges, many emerging organically from collaborative culture and faculty mentorship. Technical challenges were addressed through student-directed research, experimentation, and iterative refinement. Faculty mentorship focused on helping students develop problem-solving strategies, identify relevant resources, and understand underlying principles enabling independent troubleshooting. This approach developed students’ capacity for self-directed learning and professional problem-solving.

Learning challenges, particularly confidence issues and feeling overwhelmed, were addressed through regular weekly meetings providing opportunities for peer and faculty recognition of completed work. GitHub issue completion provided tangible evidence of progress. Weekly retrospectives helped students develop metacognitive awareness of their learning processes. The team culture of mutual support, where “everyone is still learning together,” normalized encountering unfamiliar challenges and reduced anxiety.

Project management challenges, particularly timeline adjustments, were addressed through flexible planning prioritizing learning and quality over rigid deadlines. When PCG implementation could not begin by the original September target without compromising core systems, the team collectively adjusted the timeline rather than rushing development. This reflects educational philosophy prioritizing deep learning over artificial deadline compliance and provided valuable lessons in software project management realities.

4.6 Playtesting Feedback

Internal playtesting conducted throughout development revealed technical issues including enemy spawning behavior, player damage calculation, and hitbox detection, as well as gameplay balance problems such as excessively slow player movement speed. These findings led to iterative refinements that improved overall game feel and responsiveness. The playtesting process provided students with valuable experience in systematic testing, bug documentation through GitHub issues, and understanding how technical implementation decisions impact player experience.

4.7 Comparison to Traditional Semester Projects

Preliminary observations suggest substantial qualitative differences from typical semester projects. The most significant difference is project depth and student ownership. Where five-week projects produce functional prototypes demonstrating specific concepts, the extended timeline allowed students to develop a cohesive, polished game with integrated systems, custom art, and iterative refinement based on playtesting. The project evolved into a “passion project,”

with dedication extending beyond assignment completion to genuine investment in quality.

The extended duration enabled authentic collaborative development practices difficult to cultivate in shorter projects. Students developed expertise with distributed version control, code review, and asynchronous communication. Team culture evolved from individuals on assigned tasks to a cohesive development team with shared creative vision. Faculty observations suggest semester project students rarely develop this collaborative sophistication, treating version control as a submission mechanism and maintaining transactional relationships focused on task division rather than collective creation.

Finally, the extended timeline changed the relationship between learning and production. Semester projects require students to apply existing knowledge, with limited time for acquiring new skills during the project. The 14-week duration allowed students to identify knowledge gaps during development, engage in self-directed learning, and apply newly acquired skills within the same project. This integration mirrors professional software development, where continuous learning is essential, rather than compartmentalizing learning from application. The result is deeper technical understanding, as students must implement solutions and develop foundational knowledge to understand why solutions work and how to adapt them.

5 Discussion

5.1 Interpretation of Preliminary Findings

5.1.1 Extended Timeline Impact. The 14-week intensive timeline (July 14–October 17, 2025) emerged as a critical factor differentiating this project from traditional semester-based coursework. Analysis reveals four distinct developmental phases: foundation and onboarding (weeks 1–4), during which students achieved Git/GitHub comfort and produced a minimum viable starting point; core feature development (weeks 5–8), marked by the transition from instructor-directed tasks to student-driven ownership and the emergence of peer mentorship; iteration and refinement (weeks 9–12), enabling experimentation with implementations students knew would not appear in the final product; and architectural maturity (weeks 13–14), during which students voluntarily refactored concrete classes into abstract base classes with scriptable object descendants. The onboarding phase consumed approximately 30% of the timeline in both 14-week and typical 5-week projects, but this translates to fundamentally different absolute development times: 10.5 weeks versus 3.5 weeks. Faculty mentor estimates suggest minimum timelines of 3 weeks to overcome tool anxiety, 4–5 weeks for genuine experimentation comfort, 6–7 weeks for core features, and approximately 10 weeks for meaningful iteration cycles.

The extended timeline enabled pedagogical experiences largely absent in compressed projects. Students gained psychological safety to experiment, fail, and iterate without jeopardizing project completion. The architectural maturity phase demonstrated perhaps the most significant impact: students' voluntary transition from "get it done" to "make it extensible and good" represents sophisticated software engineering thinking emerging only when students have both time and accumulated experience to recognize limitations of their initial implementations. Professional development includes

cycles of implementation, reflection, and architectural improvement, while compressed academic projects rarely progress beyond initial implementation. This manifested in students' organic emergence of distributed leadership, independent GitHub issue creation, and self-scheduled collaborative work sessions beyond required meetings.

Most critically, the extended timeline produced a psychological transformation: students began viewing the project as "theirs" rather than as an assignment to be completed. This shift in student identity relative to their work, from task-completer to creator, represents perhaps the most valuable outcome of extended project timelines. In typical 5-week projects, students rarely develop genuine passion or ownership; most effort focuses on tool familiarization and bug resolution rather than conceptual mastery or architectural thinking. This transformation in student identity directly addresses the portfolio and employability gaps motivating this research, as authentic creator engagement produces work students willingly showcase and discuss in professional contexts.

5.1.2 Experiential Learning Validation. The project's structure and progression provide substantial evidence validating Kolb's Experiential Learning Theory. Analysis of student work patterns, meeting discussions, and code evolution reveals the four-stage cycle operating at both individual and team levels throughout the 14-week timeline, with particularly clear manifestations in technical decision-making, collaborative practices, and architectural evolution. Weekly meetings followed an Agile standup pattern, creating formalized opportunities for reflective observation where students reported their work, planned next steps, and identified obstacles. Critically, discussions extended beyond status updates to include reflective questions: "What did you learn from this work?" and "What would you do differently next time?" This structured reflection transformed concrete coding experiences into conscious learning opportunities. The cycle became visible in students' evolving communication patterns: early meetings focused on task completion ("I finished the player movement"), while later meetings demonstrated reflective depth ("The collision handling worked initially but became problematic with multiple enemies; I need to rethink the knockback calculation"). This shift from reporting activities to analyzing approaches indicates internalization of reflective observation as a learning strategy.

The evolution of enemy and item systems provides a textbook example of Kolb's complete cycle. Students initially implemented enemies and items as concrete, hard-coded classes (RangedEnemy, MeleeEnemy, HealthItem, AmmoItem), a straightforward **concrete experience** that produced functional game systems. As the project progressed and the number of enemy and item types grew, students encountered practical problems: code duplication, difficulty adding new variants, and maintenance complexity. This friction prompted **reflective observation**: the initial approach worked adequately for 1–2 enemies or items but clearly would not scale. Students moved to **abstract conceptualization** by researching object-oriented design patterns, learning about abstract base classes, virtual methods, and scriptable objects. They developed a theoretical understanding of why their initial approach failed (tight coupling between data and code) and how abstraction principles could address these problems.

Finally, they engaged in **active experimentation** by implementing EnemyManager and ItemManager systems with abstract base classes and scriptable object descendants. The new architecture succeeded, validating their conceptual learning and providing a reusable pattern for future systems.

Importantly, this cycle occurred organically through authentic development needs rather than instructor-imposed exercises. Students were not told their initial approach was “wrong”; they discovered its limitations through experience, reflected on those limitations, developed theoretical understanding of alternatives, and tested new approaches. This authentic engagement with the full learning cycle distinguishes experiential learning from traditional instruction where concepts precede application. The project timeline enabled students to engage in productive failures: implementations like the initial player object, characterized by students as “thrown together to get something playable,” worked sufficiently to enable progress but were understood from the outset as temporary scaffolding. The learning outcomes from these experiences were substantial: students developed practical understanding of code refactoring, item manager architecture, abstract and virtual objects, object pooling patterns, and the relationship between rapid prototyping and production-quality code. This willingness to create temporary implementations for learning purposes reflects comfort with the experiential learning cycle: recognizing that concrete experience, even when the immediate product is disposable, generates valuable reflective and conceptual learning. The cycle also manifested at team and social levels: collaborative practices evolved from centralized faculty-mediated communication to distributed peer decision-making, and individual learning cycles became collective learning through peer knowledge sharing and mentorship.

Implications for Experiential Learning Design. The project’s validation of Kolb’s cycle suggests several design principles for experiential learning in computer science education. First, sufficient timeline duration is essential: the cycle requires time for consequences of initial implementations to manifest, prompting genuine reflection rather than superficial correction. Second, authentic problems produce more powerful learning than contrived exercises: students’ motivation to refactor enemy and item systems emerged from real scaling problems, not instructor mandates. Third, structured reflection opportunities (weekly discussions focusing on learning, not just progress) transform tacit experiences into conscious learning. Fourth, tolerance for productive failure enables the complete cycle: students must feel safe creating imperfect implementations that serve learning purposes even when not production-ready. Finally, peer collaboration accelerates cycling by providing multiple concrete experiences to reflect upon and multiple perspectives during conceptualization phases. These principles suggest that extended, authentic, collaborative projects with structured reflection create ideal conditions for experiential learning in technical domains.

5.1.3 PCG as Pedagogical Tool. The 14-week foundation phase proved essential in preparing students for the upcoming Wave Function Collapse implementation scheduled for January 2026. Students have already begun preliminary investigation into WFC algorithms through YouTube tutorials, scholarly articles, and sample code repositories, supplemented by instructor-led walkthroughs of example implementations in Lua. Critically, the extended timeline

allowed students to develop the contextual understanding necessary for effective PCG implementation beyond the technical skills (object-oriented principles, 2D array manipulation, recursion). Had PCG implementation been attempted in week 5, students would have possessed the raw technical capabilities but lacked the domain knowledge to avoid what is often called “the 100 bowls of oatmeal problem”—procedurally generating content that is technically varied but experientially indistinguishable. The foundation phase enabled students to understand what makes their specific game engaging, providing the conceptual framework necessary to design meaningful constraints for procedural generation.

The decision to implement a custom WFC algorithm rather than adopt an existing library reflects both pedagogical objectives and practical project constraints. The game’s unique tile requirements and gameplay mechanics necessitate tailored PCG solutions that off-the-shelf libraries cannot provide without substantial modification. Students are expected to develop integrated competencies across three domains: technical skills (implementing constraint satisfaction algorithms and managing stochastic systems), design skills (understanding how procedurally generated content affects gameplay balance and player experience), and systems thinking (ensuring generated content respects game-wide constraints while maintaining variety). This represents precisely the type of complex, authentic problem-solving that distinguishes extended research projects from conventional coursework, where PCG would typically be demonstrated through tutorials rather than designed, implemented, and integrated into a production game system.

5.2 Implications for CS Education

5.2.1 Scalability and Transferability. The model demonstrates clear potential for replication at similar institutions, with planned expansion to a broader summer workshop format. The next iteration will introduce students to game design fundamentals using smaller frameworks such as Love2D with Lua or Axmol with C++. The workshops will follow a structured progression: 1–2 weeks for technology familiarization and creating a playable prototype, 1–2 weeks for brainstorming and iterating on initial concepts, 1–2 weeks implementing a PCG algorithm tailored to their specific game, and remaining time devoted to polish and refinement. This condensed format maintains the essential pedagogical elements, extended timeline for authentic development, experiential learning through iteration, and technical challenge through PCG implementation, while making the model accessible to institutions with summer program infrastructure.

5.2.2 Curriculum Integration. This extended project model complements traditional computer science curricula by providing authentic contexts for applying concepts introduced across multiple courses. The game development workflow naturally integrates foundational programming concepts (variables, control structures, functions), object-oriented design principles (encapsulation, inheritance, polymorphism), data structures (arrays, lists, trees for spatial partitioning), algorithms (pathfinding, constraint satisfaction, procedural generation), software engineering practices (version control, code review, testing, documentation), and collaborative development workflows. Rather than encountering these topics as isolated exercises, students experience them as interconnected elements of

a complex system where design decisions have cascading consequences. This systems-level understanding represents a critical bridge between academic learning and professional practice that traditional course sequences struggle to provide within semester constraints.

5.2.3 Student Employability and CSU System Priorities. The project directly addresses CSU system priorities around workforce preparation and student employability through multiple mechanisms. First, the paid research assistant positions provide documented work experience in software development, directly addressing the paradox that entry-level positions require prior experience. Students transition from “no professional experience” to “paid software developer on a year-long game project.” Second, the project produces portfolio-quality artifacts demonstrating technical competence to prospective employers: a published game, GitHub repository with professional development practices, and tangible evidence of collaborative software development skills. Third, the extended mentorship and professional skill development (code review, project management, technical communication) prepare students for workplace expectations beyond pure coding ability. Finally, the model demonstrates how undergraduate research programs can serve workforce-bound students, not just those pursuing graduate education.

5.3 Limitations

As a work-in-progress study, this research faces several important limitations. The project remains incomplete at the time of writing, with full demo completion targeted for March 2026 including procedural content generation. Consequently, the findings presented here represent preliminary observations based on October 2025 data rather than conclusive results from a completed project. The small sample size, a single team of four students at one institution, limits generalizability. Follow-up studies with complete outcome data, larger sample sizes, and controlled comparisons will be necessary to validate and extend these preliminary findings.

Additional methodological constraints warrant acknowledgment. Data collection relies primarily on self-report instruments (student reflections and surveys) and faculty observations, introducing potential biases including social desirability effects and observer bias stemming from the principal investigator’s dual role as mentor and researcher. The study lacks preexisting validated instruments specifically designed for assessing experiential learning outcomes in extended technical projects, necessitating reliance on qualitative analysis of student reflections. Contextual factors further limit transferability: internal funding mechanisms supporting this work are not universally available; summer availability varies substantially across student populations, particularly for those requiring year-round employment; game development may not appeal broadly to all computer science students; and procedural content generation represents a relatively advanced topic that may not suit all skill levels. Despite these limitations, the richness of qualitative data and detailed documentation of student learning processes provide valuable insights for designing similar interventions at comparable institutions.

6 Next Steps and Future Work

6.0.1 Project Completion.

- Implement full PCG system (December 2025 - February 2026)
- Polish and public release (March-April 2026)
- Student presentations at CSUMB (May 2026)

6.0.2 Data Collection.

- Final student surveys and reflections
- Post-project interviews
- Portfolio artifact analysis
- Long-term tracking (job placement outcomes)

6.0.3 Dissemination.

- Public game release (itch.io, Steam)
- Social media campaign for playtesting
- GitHub repository documentation
- Conference presentations (CCSC, ASEE, SIGGRAPH, GDC)

6.1 Educational Material Development

A key objective is developing reusable educational materials based on lessons learned. Once the Wave Function Collapse implementation is complete (spring 2026), we will create modular lesson plans and tutorials suitable for integration into existing CS courses. These materials will include: (1) standalone PCG modules for algorithms courses covering constraint satisfaction and backtracking; (2) game development project templates for software engineering courses emphasizing collaborative workflows and version control; (3) documented case studies of student learning challenges and solutions; and (4) assessment rubrics for evaluating student work on extended game development projects. Materials will be released open-source alongside the project repository to enable replication at similar institutions. Additionally, we plan to develop a condensed summer workshop format that adapts the 14-week model for broader institutional contexts using accessible frameworks like Love2D or Axmol.

6.2 Longitudinal Study

Understanding the long-term impact of this project on student career trajectories requires longitudinal tracking beyond project completion. We will conduct follow-up surveys and interviews with student participants at 6-month, 1-year, and 2-year intervals to assess: (1) employment outcomes and job placement success; (2) perceived value of project experience during job searches and interviews; (3) application of technical and professional skills learned during the project in workplace contexts; (4) portfolio artifact usage and effectiveness in demonstrating competence to employers; and (5) sustained interest in game development or related fields. This longitudinal data will provide empirical evidence regarding the project’s effectiveness as an employability intervention and inform future iterations of the model. We will also track whether participants pursue graduate education versus direct workforce entry, providing insights into how extended research experiences serve diverse student career goals.

6.3 Future Research Directions

This work-in-progress opens several avenues for future research. First, controlled comparison studies examining learning outcomes between extended projects (14 weeks) versus traditional semester projects (5 weeks) would provide stronger empirical evidence for timeline effects, though logistical and ethical challenges complicate such designs. Second, replication at other institutions with different student populations and resources would test the model's generalizability and scalability. Third, investigating alternative technical domains beyond game development (e.g., web applications, mobile apps, embedded systems) would reveal whether observed benefits derive from extended timelines generally or from game development specifically. Fourth, examining optimal project structures for different student skill levels would inform scaffolding and mentorship strategies. Finally, developing validated assessment instruments for measuring experiential learning outcomes in extended technical projects would benefit the broader CS education research community. We anticipate iterating on this model through the planned summer workshop format, providing opportunities to systematically vary project parameters and assess their effects on learning outcomes.

7 Conclusion

This work-in-progress paper has presented a 14-week intensive summer research project designed to bridge the gap between academic preparation and industry requirements in game development. By combining experiential learning theory with authentic software development practices and novel algorithm implementation (Wave Function Collapse for procedural content generation), this project provides students with portfolio-quality artifacts and professional skills enhancing their employability.

The extended timeline addresses critical limitations of traditional semester projects, allowing students to engage with the complete software development lifecycle, implement complex algorithms, and produce polished, demonstrable work. Preliminary observations suggest that this model effectively combines rigorous computer science education with practical career preparation.

This work contributes to the growing body of research on experiential learning in CS education, game-based learning pedagogies, and the integration of industry-relevant practices into undergraduate curricula. The project's focus on procedural content generation as a pedagogical tool represents a novel contribution, addressing a gap in the literature regarding PCG's potential for teaching core CS concepts. As an explicit employability intervention aligned with CSU system priorities, this work also demonstrates how summer research programs can be adapted to serve students entering the technology workforce directly after graduation, complementing traditional research experiences oriented toward graduate school preparation.

Future work will complete the project, collect comprehensive outcome data, develop reusable educational materials, and conduct longitudinal tracking of student career trajectories. The lessons learned and materials produced will be disseminated to enable replication at similar institutions, contributing to a broader transformation of CS education toward more authentic, industry-aligned learning experiences.

Acknowledgments

This work was supported by the California State University, Monterey Bay Research Scholarship and Creative Activity (RSCA) grant program. We gratefully acknowledge the contributions of our student research team members whose dedication, creativity, and passion for game development made this project possible. Their technical implementations, artistic contributions, and thoughtful reflections on their learning experiences form the foundation of this research. We also thank the CSUMB Computer Science and Information Technology department for supporting this extended research experience.

References

- [1] Matheus de Almeida Rodrigues, Licínio Roque, and António Ramires Fernandes. 2022. The effectiveness of gamification in programming education: Evidence from a meta-analysis. 182 (2022), 104455. doi:10.1016/j.cae.2022.104455 Publisher: Elsevier.
- [2] Christo Dichev and Darina Dicheva. 2017. Gamifying education: What is known, what is believed and what remains uncertain: a critical review. 14, 1 (2017), 9. doi:10.1186/s41239-017-0042-5 Publisher: Springer Open.
- [3] James Paul Gee. 2003. *What video games have to teach us about learning and literacy*. Palgrave Macmillan.
- [4] Zachary Kissel and Christopher Stuetzle. 2020. Experiential learning framework for smaller computer science programs. 35, 8 (2020), 66–75.
- [5] David A. Kolb. 1984. *Experiential learning: experience as the source of learning and development*. Prentice-Hall.
- [6] Abdullah Konak, Tricia K. Clark, and Mahdi Nasereddin. 2014. Using Kolb's Experiential Learning Cycle to improve student learning in virtual computer laboratories. 72 (2014), 11–22. doi:10.1016/j.compedu.2013.10.013
- [7] Research {and} Markets Ltd. [n.d.]. *United States Video Game Market Size & Forecast to 2033*. <https://www.researchandmarkets.com/reports/5546210/united-states-video-game-market-by-category-age>
- [8] Mahdi Farrokhi Maleki and Richard Zhao. 2024. Procedural Content Generation in Games: A Survey with Insights on Emerging LLM Integration. 20, 1 (2024), 167–178. arXiv:2410.15644 [cs] doi:10.1609/aiide.v20i1.31877
- [9] Elisa D. Mekler, Florian Brühlmann, Alexandre N. Tuch, and Klaus Opwis. 2017. Towards understanding the effects of individual gamification elements on intrinsic motivation and performance. 71 (2017), 525–534. doi:10.1016/j.chb.2015.08.048 Publisher: Elsevier.
- [10] Thomas Howard Morris. 2020. Experiential learning – a systematic review and revision of Kolb's model. 28, 8 (2020), 1064–1077. doi:10.1080/10494820.2019.1570279 Publisher: Routledge _eprint: <https://doi.org/10.1080/10494820.2019.1570279>
- [11] Seymour Papert. 1980. *Mindstorms: Children, computers, and powerful ideas*. (1980). ISBN: 9780465046270 Place: New York, NY, USA Publisher: Basic Books.
- [12] Maja Videnovik, Tone Vold, Linda Kjønnig, Ana Madevska Bogdanova, and Vladimir Trajkovik. 2023. Game-based learning in computer science education: a scoping literature review. 10, 1 (2023), 54. doi:10.1186/s40594-023-00447-2 Publisher: Springer.
- [13] Hongyu Zeng, Ronghui Huang, Junfeng Li, Jiahao Wu, Xiaoqing Gu, Jian Wang, and Kinshuk. 2024. Exploring the impact of gamification on students' academic performance: a comprehensive meta-analysis of studies from the year 2008 to 2023. 55, 2 (2024), 567–589. doi:10.1111/bjet.13471 Publisher: Wiley-Blackwell.

A Online Resources

The project repository is publicly available at https://github.com/daclink/MBAG_RogueDreams.